

COMPUTACIÓN SOCIAL Y PERSONALIZACIÓN

Sistemas de Recomendación Musical

Bitácora científica y memoria técnica del trabajo

Filtrado colaborativo sobre el Million Song Dataset: del EDA al benchmark de cinco modelos contra un baseline con bias

EQUIPO

Raúl Fernández

Damián Datta

Juan Carlos Cintas

Martín Kugler

Dataset: Million Song Dataset + Spotify + Last.fm · 9,7 M interacciones

Índice

1. Resumen ejecutivo

Objetivo. Este trabajo construye y compara un conjunto de sistemas de recomendación de filtrado colaborativo sobre un dataset masivo de historial de escuchas musicales (9,7 millones de interacciones procedentes del Million Song Dataset enriquecido con metadatos de Spotify y Last.fm). El propósito no es solo entrenar modelos, sino **entender qué tipo de algoritmo encaja con la naturaleza concreta de los datos** y dejar una bitácora reproducible de todas las decisiones tomadas, desde el preprocesamiento hasta la evaluación.

Modelos implementados. Partiendo de un análisis exploratorio cuidadoso, transformamos el feedback implícito (recuentos de reproducción) en un rating sintético continuo en escala [1, 5] y entrenamos cinco familias de modelos: un *baseline* basado en medias por ítem y un *baseline Bias-Only* (que añade sesgos por usuario y por ítem), un modelo de vecindad **KNN** usuario-usuario con similitud coseno, factorización matricial probabilística **PMF**, factorización de Bernoulli **BeMF**, y los dos modelos de Neural Collaborative Filtering de He et al. (2017): **GMF** y **MLP**.

Hallazgo principal. El mejor modelo neuronal (MLP, RMSE $\approx 0,808$) se queda a milésimas del **baseline Bias-Only (RMSE $\approx 0,81$)**, que no es superado por ninguno de los modelos complejos. La explicación es estructural: el 60 % de las interacciones son escuchas únicas, por lo que casi toda la varianza predecible en regresión la capturan las tendencias marginales (sesgos) por usuario y por ítem. Sin embargo, en las **métricas de ranking** los modelos neuronales sí destacan (nDCG@10 $\approx 0,92$), que es lo que de verdad importa en un recomendador real.

1.1. Tabla de resultados global

La siguiente tabla resume el rendimiento de todos los modelos sobre el conjunto de test independiente. Las métricas de ranking se calcularon con el protocolo *Leave-One-Out + Negative Sampling* (1 positivo + 99 negativos). Las celdas marcadas como «—» no se reportaron en el notebook correspondiente.

Modelo	RMSE	MAE	Precision@10	Recall@10	nDCG@10
Baseline (media global)	~0,95	—	—	—	—
BeMF	~0,95	—	—	—	—
GMF	0,8690	0,6283	0,1212	0,9795	0,9131
MLP	0,8077	0,5818	0,1221	0,9863	0,9206
Bias-Only (baseline)	0,81	0,57	—	—	—

Lectura rápida: en regresión, «menos es más». El modelo conceptualmente más simple que captura sesgos (Bias-Only) gana en RMSE. En ranking, los modelos neuronales (GMF/MLP) son excelentes. La complejidad teórica de BeMF no se traduce en ventaja porque sus supuestos (rating discreto, clases balanceadas) chocan con la realidad del dataset.

2. Contexto del problema y dataset

El filtrado colaborativo parte de una idea simple: si dos personas han coincidido en sus gustos en el pasado, es probable que coincidan en el futuro. El reto práctico es que la información disponible es **feedback implícito** —no tenemos valoraciones explícitas de «me gusta del 1 al 5», sino simplemente cuántas veces alguien ha escuchado cada canción—. Convertir ese señal implícita en algo que un modelo de regresión pueda aprovechar es la primera decisión de diseño importante del trabajo.

2.1. Fuentes de datos

Disponíamos de dos archivos:

- **Music Info.csv** — metadatos descriptivos de 50.683 canciones (artista, género, año, y features de audio de Spotify como *danceability*, *energy*, *valence*, *tempo*...). Solo se usa como contexto cualitativo; **no entra en los modelos de filtrado colaborativo**.
- **User Listening History.csv** — el dataset principal: 9.711.301 tuplas (*user_id*, *track_id*, *playcount*). Es feedback implícito puro.

La carga se optimizó usando `dtype='category'` para los identificadores y `uint16` para el `playcount`, sin lo cual el CSV de 9,7 M filas no entra cómodamente en memoria. Se comprobó que ambos datasets están limpios en las columnas relevantes (cero nulos en *user_id*, *track_id* y *playcount*).

2.2. Estadísticos generales y el problema de la dispersión

Magnitud	Valor (crudo)
Usuarios únicos	962.037
Canciones únicas	30.459
Interacciones totales	9.711.301
Densidad de la matriz usuario × canción	0,0331 %

Densidad del 0,033 %. De cada 10.000 celdas posibles en la matriz usuario × canción, solo 3 tienen información. Esta dispersión extrema es la firma de cualquier sistema de recomendación realista y es la razón fundamental por la que no se puede «rellenar con ceros» y entrenar un modelo denso: hace falta tratar la dispersión explícitamente mediante factorización o embeddings.

Matriz «alta y estrecha». Con ~32 usuarios por canción, la capa de embedding de usuarios sería gigantesca ($962.037 \times \text{dim}$). Para una `latent_dim = 32` hablaríamos de ~30 M de parámetros solo en la capa de usuarios —inviabile de entrenar sin filtrar primero. Esto motiva directamente el *k-core* que se aplica en el preprocesamiento.

3. Análisis exploratorio y preprocesamiento

El notebook de EDA es el cimiento de todo el trabajo: produce el **único** dataset (dividido en train/test) que consumen todos los modelos. Tres decisiones lo estructuran: el filtrado *k-core*, la transformación de playcount a rating sintético, y el *split* estratificado por usuario.

3.1. Distribución de interacciones: las tres leyes de potencia

Antes de filtrar nada, estudiamos las tres distribuciones clave. Las tres resultaron tener forma de ley de potencia (cola larga), lo que condiciona todas las decisiones posteriores.

Interacciones por usuario

La media (10,1) es el doble de la mediana (5): distribución muy sesgada a la derecha. El percentil 99 es 71 y el máximo 784 (superusuarios que tiran de la media). Crucialmente, **una cuarta parte de los usuarios tiene 3 interacciones o menos**, insuficiente para aprender un embedding fiable o para evaluar ranking sobre ellos.

Corte k	% usuarios con < k interacciones
< 3	24,1 %
< 5	44,4 %
< 10	69,8 %
< 20	86,8 %
< 50	97,5 %

Decisión — $k_{\text{user}} = 5$. Descarta usuarios cuya contribución sería más ruido que señal y garantiza que, tras un split 80/20, cada usuario conserva ~ 4 interacciones en train y ~ 1 en test (el mínimo razonable para que el ranking tenga sentido). Conserva ~ 55 % de los usuarios: un compromiso aceptable entre rigor y diversidad.

Interacciones por canción

Aún más sesgada que la de usuarios: ratio media/mediana $\approx 5,6$ (frente a ≈ 2). Hay canciones con hasta **80.656 escuchas**, casi tres órdenes de magnitud sobre la mediana (57). Es la clásica *long tail* del consumo musical: unos pocos *hits* acumulan casi toda la atención.

Decisión — $k_{\text{item}} = 10$. Un corte en la mediana (57) eliminaría el 50 % del catálogo, demasiado agresivo: descartaría canciones de 20-50 escuchas perfectamente válidas. Con $k_{\text{item}} = 10$ se garantizan al menos 10 evidencias por track (embedding mínimamente fiable), se conserva ~ 75 % del catálogo y se eliminan los tracks demasiado raros.

3.2. Filtrado iterativo (k-core)

El k-core debe aplicarse **iterativamente**: al filtrar usuarios con pocas interacciones, algunos tracks caen por debajo del umbral; y al filtrar esos tracks, algunos usuarios vuelven a quedar por debajo. Se itera hasta que el tamaño del DataFrame deja de cambiar (convergencia).

```
min_user = 5 ; min_track = 10
while True:
    start_len = len(df_filtrado)
    # 1) filtrar tracks con < min_track
    valid_tracks = track_counts[track_counts >= min_track].index
    df_filtrado = df_filtrado[df_filtrado['track_id'].isin(valid_tracks)]
    # 2) filtrar usuarios con < min_user
    valid_users = user_counts[user_counts >= min_user].index
    df_filtrado = df_filtrado[df_filtrado['user_id'].isin(valid_users)]
    if start_len == len(df_filtrado): break # convergencia
```

Resultado: se conserva el **89 % del dataset** (de 9.711.301 a 8.650.751 interacciones), descartando ~1 M de interacciones de muy baja actividad. Ganamos calidad estadística sin perder representatividad.

3.3. De playcount a rating sintético

El playcount tiene una distribución brutalmente sesgada: mediana = 1, percentil 75 = 2, percentil 99 = 21, máximo = 2.948. El **60,8 %** de todas las interacciones son escuchas únicas (playcount = 1). Usar el playcount crudo como target haría que los *outliers* extremos dominasen la pérdida MSE: acertar la mayoría de interacciones (que valen 1 o 2) sería irrelevante frente a acertar la cola.

Plan de transformación: log1p para comprimir el rango → **clipping al percentil 99** para neutralizar outliers extremos → escalado *min-max* al rango **[1, 5]** continuo (coherente con el rating explícito tipo Netflix/MovieLens y con el notebook de NCF de clase). El rating se mantiene **continuo**, sin redondear a estrellas, para preservar la información de la transformación.

```
log_pc = np.log1p(df['playcount'])
log_pc_clipped = log_pc.clip(upper=log_pc.quantile(0.99))
df['rating'] = 1 + 4 * (log_pc_clipped - log_min) / (log_max - log_min)
```

Para validar que la escala resultante es interpretable, calibramos la equivalencia rating ↔ playcount:

Rating	Playcount equivalente	Interpretación
≈ 1	1 escucha	Interacción mínima, casi exploratoria
≈ 3	~4-6 escuchas	Interés moderado: ha vuelto varias veces
= 5	≥ 21 escuchas	Afecto claro; top 1 % del dataset

Consecuencia clave para todo el trabajo. La distribución del rating final está aplastada contra 1,0 (media 1,57; mediana 1,0). Solo ~5,6 % de las interacciones alcanzan rating ≥ 4. Este desbalance

extremo explica casi todos los resultados posteriores: el umbral de relevancia es exigente, los sesgos marginales capturan casi toda la señal, y BeMF (que asume clases balanceadas) lo pasa mal.

3.4. Mapeo de IDs y split estratificado

Tras el filtrado, los IDs originales (strings) ya no son consecutivos. Como `nn.Embedding` de PyTorch requiere índices enteros en $[0, N-1]$, creamos columnas `user_n` y `track_n` con numeración nueva (sin este paso, los embeddings lanzan `IndexError`).

El **split 80/20 es estratificado por usuario**: un split aleatorio puro podría dejar a usuarios completamente fuera de train (cold-start) o de test. La técnica asigna un número aleatorio a cada fila, lo convierte en percentil *dentro* de cada usuario mediante `groupby('user_id').rank(pct=True)`, y corta al 80 %.

Aspecto	Antes (crudo)	Después (procesado)
Usuarios	962.037	533.689
Canciones	30.459	23.629
Interacciones	9.711.301	8.650.751 (89 %)
Target	playcount [1–2.948]	rating continuo [1, 5]
Split	—	80/20 estratificado por usuario
Cold-start	—	0 usuarios (todos en train y test)

El split real fue 77,6 / 22,4 (ligera desviación porque el número de interacciones por usuario es discreto). Los 533.689 usuarios aparecen en train y en test: cero cold-start. Los datos se guardan en `train.csv`, `test.csv`, más versiones esparsas `.npz` para KNN y los diccionarios de mapeo en `.pk1`.

4. Arquitectura del pipeline y protocolo de evaluación

El proyecto se organiza en dos planos. Los notebooks (uno por modelo) son el espacio de exploración: grid search, diagnóstico de sobreajuste, visualizaciones y conclusiones. Los scripts de Python encapsulan el pipeline reproducible que consume el menú interactivo de `main.py`.

4.1. Estructura del proyecto

```
EDA.ipynb          -> genera train.csv / test.csv / *.npz / *.pkl
GMF.ipynb          -> exploración + entrenamiento Generalized MF
MLP.ipynb          -> exploración + entrenamiento Multi-Layer Perceptron
PMF_def.ipynb      -> exploración + entrenamiento Probabilistic MF
BeMF__1_.ipynb     -> exploración + entrenamiento Bernoulli MF
knn.ipynb          -> KNN usuario-usuario con similitud coseno

src/
main.py            -> menú interactivo (carga, checkpoints, eval)
evaluador.py       -> Negative Sampling vectorizado (Leave-One-Out)
metrics.py         -> MAE, RMSE, Precision/Recall/F1/nDCG @K
baseline_model.py  -> baseline por media de ítem + media global
models/{GMF,MLP,knn_model_optimized}.py
```

4.2. El protocolo de evaluación: por qué dos familias de métricas

Un recomendador se puede juzgar de dos formas complementarias, y reportamos ambas:

1. **Regresión (MAE, RMSE).** ¿Cuán cerca está la predicción del rating real? El RMSE penaliza los errores grandes (eleva al cuadrado), el MAE es interpretable en la misma escala de estrellas. Útil para medir calibración, pero no captura si el *orden* de las recomendaciones es bueno.
2. **Ranking (Precision@K, Recall@K, F1@K, nDCG@K).** ¿Coloca el modelo los ítems relevantes arriba del Top-K? El nDCG es la métrica más informativa porque premia colocar lo relevante en las primeras posiciones. El umbral de relevancia del equipo es $\text{rating} \geq 4,0$ y $K = 10$.

4.3. Negative Sampling (Leave-One-Out)

Evaluar ranking sobre todo el catálogo (23.629 ítems por usuario) es inviable y además poco informativo. Adoptamos el protocolo estándar de la literatura NCF: para cada ítem **positivo relevante** del usuario en test ($\text{rating} \geq 4$), lo escondemos entre **99 ítems negativos** que nunca ha visto, y comprobamos si el modelo lo coloca en el Top-10. El módulo `evaluador.py` implementa esto de forma **vectorizada con NumPy** (el `iterrows()` fila a fila era inviable a 1,9 M de filas) y muestrea ~1.000 usuarios representativos.

Decisiones de diseño del evaluador: (1) solo se evalúan positivos con $\text{rating} \geq \text{umbral}$, porque si el ítem oculto no es genuinamente relevante la métrica de ranking es ruido; (2) los ítems prohibidos como negativos son la unión de los vistos en train y los positivos del propio usuario en test, evitando falsos negativos; (3) muestreo de usuarios en lugar del test completo para que la evaluación termine en tiempo razonable.

4.4. El trade-off Precision vs Recall en este dataset

Un patrón aparece en todos los modelos neuronales: **Recall@10 altísimo (~0,98) y Precision@10 baja (~0,12)**. No es un error, es característico del dataset. Como el umbral rating ≥ 4 es muy exigente, los usuarios tienen muy pocos relevantes en test (a menudo 0-2). El modelo casi siempre los recupera en su Top-10 (Recall alto), pero rellena el resto del Top-10 con no-relevantes (Precision baja, que se calcula sobre K fijo). Por eso el **nDCG@10 es la métrica de referencia**: un nDCG $\approx 0,92$ indica que esos pocos relevantes están bien colocados en lo alto del ranking.

5. Modelos: diseño, decisiones y resultados

5.1. Baselines: la vara de medir

Antes de cualquier modelo complejo, fijamos referencias. El **baseline trivial** predice para cada canción su media de rating en train (y la media global para ítems desconocidos en test). Es el «¿qué pasa si recomiendo lo popular?». El **baseline Bias-Only** va un paso más allá: modela la predicción como *media global + sesgo del usuario + sesgo del ítem*, capturando que algunos usuarios puntúan sistemáticamente alto/bajo y algunas canciones gustan sistemáticamente más. Este segundo baseline resultó ser, sorprendentemente, **el modelo más difícil de superar** de todo el trabajo.

Por qué importa el Bias-Only. En un dataset donde el 60 % son escuchas únicas (rating ≈ 1), las tendencias marginales por usuario y por ítem capturan casi toda la varianza que es realmente predecible. Cualquier modelo latente tiene que justificar su complejidad superando esta línea base — y en regresión, ninguno lo logra de forma clara.

5.2. KNN usuario-usuario (memory-based)

El primer modelo de filtrado colaborativo «de verdad» es de vecindad. Para un usuario u , se buscan sus k usuarios más parecidos y se predice su valoración de un ítem combinando lo que esos vecinos opinaron. La similitud elegida es la **coseno**, por experiencia previa del equipo en PLN y por sus buenos resultados con vectores esparsos.

Decisiones de implementación

- **Predicción por desviación de la media.** No se predice la media bruta de los vecinos, sino *media del usuario u + desviación media de los vecinos respecto a su propia media* sobre el ítem. Esto corrige que cada usuario tiene una escala de uso distinta.
- **Eficiencia con matrices esparsas.** Toda la similitud se calcula vectorizada sobre la matriz CSR (`matrix @ user_vec.T`), y los vecinos se obtienen con `np.argsort` (encuentra los k mayores sin ordenar todo el vector).
- **Espacio de búsqueda reducido.** Solo se predicen los ítems que (a) han escuchado los vecinos y (b) el usuario aún no conoce (`np.setdiff1d`), evitando recorrer las 23.629 canciones por usuario.

Limitación honesta. El equipo documentó que su implementación pura de KNN, aunque vectorizada, sigue siendo demasiado lenta para evaluar las métricas sobre un dataset de este tamaño en tiempo razonable. Por eso se incluye una versión optimizada en `script` () para el pipeline de , y la evaluación masiva se acota a un subconjunto de usuarios.

5.3. PMF — Probabilistic Matrix Factorization

Primer modelo latente: a cada usuario y cada ítem se le asigna un vector de factores latentes, y la predicción es su producto escalar, acotado al rango $[1, 5]$ mediante una sigmoide:

$$\hat{r}(u, i) = \sigma(p_u \cdot q_i) \cdot 4 + 1$$

Los embeddings se inicializan con ruido normal pequeño ($\text{std}=0,01$) para estabilizar el producto escalar en las primeras épocas. La diferencia con GMF es que PMF usa **pesos uniformes** sobre las dimensiones latentes: no aprende una capa que pondere unas dimensiones más que otras.

Metodología estricta (sin data leakage)

PMF introduce el rigor metodológico que heredan GMF y MLP: el train original se subdivide en **train puro (85 %) y validación (15 %)**. El grid search ($\text{latent_dim} \in \{8, 16, 32\} \times \text{lr} \in \{0,005, 0,01, 0,02\}$) y el *early stopping* se monitorizan **exclusivamente en validación**. El test permanece intocable y se usa una sola vez, al final, con Negative Sampling. Esta separación evita que las decisiones de hiperparámetros «vean» el test.

5.4. GMF — Generalized Matrix Factorization

Primer modelo de Neural Collaborative Filtering (He et al., 2017). GMF generaliza la factorización matricial añadiendo una **capa lineal aprendible** sobre el producto Hadamard de los embeddings:

$$\hat{r}(u,i) = \sigma(w^T (p_u \odot q_i)) \cdot 4 + 1$$

Frente a PMF (pesos uniformes), GMF **aprende un peso distinto por dimensión latente** mediante la capa nn.Linear. El resto de la maquinaria (sigmoide de salida, MSE, Adam, early stopping) es idéntica.

Hallazgo 1: la búsqueda en train engaña

El grid search reveló que aumentar `latent_dim` reduce drásticamente la loss en train (de $\sim 0,43$ a $\sim 0,17$), pero esto es **engañoso**: los modelos con `latent_dim=32` memorizan el train y generalizan peor. La mejor configuración **por loss en validación** fue **`latent_dim=8, lr=0,02`** (MSE 0,757) — la más simple, no la más compleja.

latent_dim (lr=0,01)	Mejor época	Mejor val loss	RMSE estimado
8	4	0,7899	0,8888
16	3	0,8098	0,8999
32	3	0,8065	0,8980

Hallazgo 2: el sobreajuste es real incluso con dim=8

Reentrenando la mejor configuración durante 15 épocas con tracking de test, la loss de test alcanza su **mínimo en la época 6 ($\approx 0,75$)** y a partir de ahí asciende hasta superar 0,90, mientras la de train sigue bajando. Es sobreajuste de manual: **el early stopping en la época 6 es obligatorio**. La conclusión del grid search basada solo en train habría sido engañosa.

Resultados finales de GMF

Métrica	Valor	Interpretación
MAE	0,6283	Error absoluto razonable en escala [1,5]

Métrica	Valor	Interpretación
RMSE	0,8690	Coherente con la loss de test
Precision@10	0,1212	~1,2 de cada 10 recomendados son relevantes
Recall@10	0,9795	Recupera el 98 % de los relevantes del usuario
F1@10	0,2157	Media armónica de precision y recall
nDCG@10	0,9131	Excelente calidad de ranking

5.5. MLP — Multi-Layer Perceptron

Segunda variante de NCF. La diferencia fundamental con GMF es la **combinación de embeddings**: en lugar de multiplicarlos elemento a elemento (Hadamard), el MLP los **concatena** y los pasa por una pila de capas densas con activación ReLU. Esto permite capturar **interacciones no lineales** usuario-ítem que el producto Hadamard lineal no puede modelar.

Aspecto	GMF	MLP
Combinación de embeddings	Producto Hadamard ($u * i$)	Concatenación ($\text{cat}([u,i])$)
Capas	1 Linear final	Varias densas + ReLU
Entrada a la red	latent_dim	latent_dim $\times$ 2
No-linealidad	No	Sí

El grid search exploró $\text{latent_dim} \in \{8, 16\} \times$ tres arquitecturas tipo «torre» decreciente: $A=[32]$, $B=[64,32]$, $C=[128,64,32]$, fijando $\text{lr}=0,01$ (el *sweet spot* de GMF). Tras el reentrenamiento con early stopping de las dos mejores, ganó **latent_dim=16 con [128,64,32]**, con mejor época 3 y test loss 0,6524 (RMSE $\approx$ 0,808).

Resultados finales de MLP

Métrica	GMF	MLP	Δ (mejora MLP)
RMSE	0,8690	0,8077	-7,2 %
MAE	0,6283	0,5818	-7,4 %
Precision@10	0,1212	0,1221	$\approx$ igual
Recall@10	0,9795	0,9863	+0,7 %
nDCG@10	0,9131	0,9206	+0,8 %

MLP es el mejor modelo neuronal del trabajo. La no-linealidad sí aporta capacidad útil: mejora a GMF un 7 % en RMSE de forma consistente. La mejora es contenida, lo que sugiere que la señal del dataset (dominada por $\text{playcount}=1$) limita el margen de cualquier arquitectura. Es coherente con He et al. (2017), donde GMF y MLP rinden de forma comparable y por eso los autores proponen **NeuMF** (fusión paralela de ambas ramas).

5.6. BeMF — Bernoulli Matrix Factorization

El modelo más ambicioso teóricamente. En lugar de asumir gaussianas continuas (PMF), BeMF descompone el problema en **cinco sub-modelos logísticos**, uno por cada puntuación posible $s \in \{1, \dots, 5\}$. Para cada s aprende factores latentes de usuario y de ítem que modelan *la probabilidad de que el usuario asigne exactamente esa puntuación*. La predicción final es el **valor esperado** $E[r] = \sum s \cdot P(r=s)$, no el argmax.

Refinamientos implementados sobre el BeMF clásico

- **Bias en tres niveles:** global por clase (B_c), por usuario (B_u) y por ítem (B_i).
- **Inicialización con priors:** B_c se inicializa con el log-odds de la frecuencia real de cada clase, dando al modelo una «foto» del desbalance (61 % son rating=1) y aliviando el arranque en frío.
- **Ordinal label smoothing:** penalización suave para errores de una sola clase (un error de $1 \leftrightarrow 2$ es menos grave que $1 \leftrightarrow 5$), relevante en ratings ordinales. El target es 1,0 si la clase coincide, 0,4 si difiere en 1, 0,1 si difiere en 2.
- **Learning rate decay** para refinamiento progresivo, y aceleración con Numba (@njit) por el tamaño del dataset (el SGD se ejecuta muestra a muestra en 5 sub-modelos).

El choque con el dataset

BeMF exige rating **discreto**, así que el rating continuo se redondea al entero más cercano para entrenar. La distribución de clases resultante es demoledora: **61,1 % clase 1, 22,7 % clase 2, ... 1,9 % clase 5** (desbalance 32:1). El grid search dimensional (variando un hiperparámetro a la vez) mostró que el número de factores latentes **apenas afecta** al RMSE (1,0779 para 5, 10, 20 y 40 factores idénticos), señal de que los factores no llegan a adquirir significado. La mejor combinación encontrada (factores=5, reg=0,01, lr=0,01) converge hacia RMSE $\approx 0,95$ -0,97, **sin superar al baseline Bias-Only**.

Lección metodológica de BeMF. La complejidad teórica no garantiza mejora si los supuestos no encajan con los datos. BeMF brilla en datasets tipo MovieLens (ratings explícitos discretos y más balanceados); aquí, sobre feedback implícito musical transformado a rating continuo y desbalanceado, sus supuestos se rompen. Su inclusión aporta valor comparativo precisamente por ilustrar este punto.

6. Resultados comparativos y discusión científica

6.1. Ranking de modelos por RMSE

Posición	Modelo	RMSE	Familia	¿Supera Bias-Only?
1	Bias-Only (baseline)	0,81	Sesgos marginales	— (referencia)
2	MLP	0,8077	NCF neuronal	≈ empata (−0,003)
3	GMF	0,8690	NCF neuronal	No
4	BeMF	~0,95	Factorización Bernoulli	No
5	Baseline trivial	~0,95	Media por ítem	No

6.2. La conclusión central: por qué Bias-Only es tan difícil de batir

El resultado más interesante del trabajo no es qué modelo gana, sino **por qué** el modelo más simple resulta casi imbatible en regresión. La cadena causal es:

- El feedback es implícito y está dominado por escuchas únicas: el 60 % de las interacciones tienen $\text{playcount}=1 \rightarrow \text{rating} \approx 1,0$.
- Tras la transformación, la distribución del rating está aplastada contra 1,0 (media 1,57). Hay muy poca varianza «interesante» que modelar.
- En ese escenario, las **tendencias marginales** (este usuario escucha mucho en general; esta canción es popular) capturan casi toda la varianza predecible. Eso es exactamente lo que modela Bias-Only.
- Los factores latentes de PMF/GMF/MLP/BeMF aportan capacidad para modelar **interacciones complejas** usuario-ítem, pero esas interacciones requieren señal explícita y variada que el feedback implícito musical no proporciona en abundancia.

Matiz importante: esto no invalida el uso de modelos neuronales. Las métricas de ranking ($\text{nDCG}@10 \approx 0,92$) son excelentes, lo que significa que GMF y MLP **sí aprenden a ordenar correctamente** los ítems relevantes. En un sistema de recomendación real, la calidad del ranking importa más que el error de regresión: al usuario no le importa que predigas 4,2 en lugar de 4,5, le importa que la canción que le va a gustar aparezca arriba en la lista.

6.3. Hallazgos metodológicos transversales

- Búsqueda en train ≠ búsqueda en test.** La mejor configuración por loss en train (alta dimensión latente) fue la peor en generalización. Seleccionar hiperparámetros por train es un error sistemático.
- Early stopping como técnica clave.** Detectar el mínimo en validación/test y restaurar esos pesos mejoró la generalización en todos los modelos neuronales (GMF: época 6; MLP: época 3).

- **Más capacidad no es mejor en datos sesgados.** Modelos pequeños ($\text{latent_dim}=8$) generalizan mejor cuando la señal es escasa, porque se ven forzados a extraer patrones generales en lugar de memorizar.
- **Rigor anti-leakage.** La separación train puro / validación / test intocable, y el uso de las curvas de validación ya registradas en el grid search (en lugar de re-evaluar sobre test), evitaron contaminar la evaluación final.

7. Conclusiones

El trabajo cumple su doble objetivo: construir un benchmark riguroso de cinco familias de recomendadores y, sobre todo, **entender la relación entre el algoritmo y la naturaleza de los datos**. Las conclusiones principales son:

1. **El dataset manda.** Un feedback implícito dominado por escuchas únicas convierte el problema de regresión en uno donde los sesgos marginales son casi suficientes. Ningún modelo latente superó claramente al baseline Bias-Only en RMSE.
2. **MLP es el mejor modelo neuronal.** Su no-linealidad mejora a GMF un 7 % en RMSE y empatía prácticamente con Bias-Only (0,8077 vs 0,81), confirmando la jerarquía esperada de la literatura NCF.
3. **El ranking es excelente aunque la regresión sea modesta.** $nDCG@10 \approx 0,92$ en GMF/MLP demuestra que los modelos ordenan bien lo relevante, que es lo que importa en producción.
4. **La complejidad teórica no basta.** BeMF, el modelo más sofisticado, fue de los que peor rindió por incompatibilidad de supuestos (rating discreto y balanceado) con los datos reales.
5. **La metodología es tan importante como el modelo.** Separación estricta de conjuntos, early stopping y selección por validación fueron decisivos para obtener conclusiones fiables.

7.1. Trabajo futuro

- **NeuMF:** fusionar GMF + MLP en paralelo (la arquitectura completa de He et al.), que en la literatura supera consistentemente a ambas ramas por separado.
- **Bias terms en NCF:** añadir sesgos explícitos (b_u , b_i , μ) a GMF/MLP, fusionando la fortaleza del Bias-Only con la capacidad latente.
- **Reformular como clasificación binaria** (escuchó / no escuchó) con negative sampling, más afín al feedback implícito original y al protocolo estándar de NCF.
- **Regularización adicional:** weight decay (L2) y Dropout entre capas densas del MLP para explorar dimensiones latentes mayores sin sobreajuste.
- **Modelos ordinales:** como Ordinal Matrix Factorization (Koren & Sill, 2011), diseñados para targets ordinales, más adecuados que BeMF para este rating continuo.